C++ COMPLETE REFERENCE LIBRARY • MODULE 13

13 CONSTRUCTORS AND DESTRUCTORS — Complete Theory & Study Book

Comprehensive Offline Theory, Memory Diagrams, Classroom C++ Programs, Practice Questions & Solutions

Module Table of Contents

1. **Module Overview & Study Guide** (README)
2. **Theory Chapter:** 01 constructors types and initializer lists
3. **Theory Chapter:** 02 copy constructor shallow vs deep copy
4. **Theory Chapter:** 03 destructors and rule of three five zero
5. **Visual Memory & Concept Diagrams** (1 Vector Diagram)
6. **Annotated C++ Source Programs** (4 Programs)
7. **Practice Question Bank:** conceptual questions
8. **Practice Question Bank:** debugging questions
9. **Practice Question Bank:** predict the output
10. **Practice Question Bank:** programming questions
11. **Step-by-Step Solutions:** conceptual solutions
12. **Step-by-Step Solutions:** output solutions
13. **Step-by-Step Solutions:** programming solutions

13 — Constructors and Destructors

This module covers Default, Parameterized, Copy, and Move constructors, Member Initializer Lists, Shallow Copy vs. Deep Copy, the Rule of Three/Five/Zero, and Destructors.

Directory Contents

- **Theory (`01_THEORY/`)**:
  - `01_constructors_types_and_initializer_lists.md` (./01_THEORY/01_constructors_types_and_initializer_lists.md)
  - `02_copy_constructor_shallow_vs_deep_copy.md` (./01_THEORY/02_copy_constructor_shallow_vs_deep_copy.md)
  - `03_destructors_and_rule_of_three_five_zero.md` (./01_THEORY/03_destructors_and_rule_of_three_five_zero.md)
- **Programs (`02_PROGRAMS/`)**:
  - `01_default_parameterized_and_copy_ctor.cpp` (./02_PROGRAMS/01_default_parameterized_and_copy_ctor.cpp) (Category A)

  - `02_deep_copy_classroom_buffer.cpp` (./02_PROGRAMS/02_deep_copy_classroom_buffer.cpp) (Category A)
  - `03_copy_ctor_by_value_compile_error.cpp` (./02_PROGRAMS/03_copy_ctor_by_value_compile_error.cpp) (Category B)
  - `04_member_initializer_list_const_ref.cpp` (./02_PROGRAMS/04_member_initializer_list_const_ref.cpp) (Category A)
- **Practice Questions (`03_PRACTICE_QUESTIONS/`)** & **Detailed Solutions (`04_DETAILED_SOLUTIONS/`)**
- **Diagrams (`05_DIAGRAMS/`)**: `diagram_01_shallow_vs_deep_copy.svg` (./05_DIAGRAMS/diagram_01_shallow_vs_deep_copy.svg)

01 — Constructors, Constructor Overloading, and Member Initializer Lists

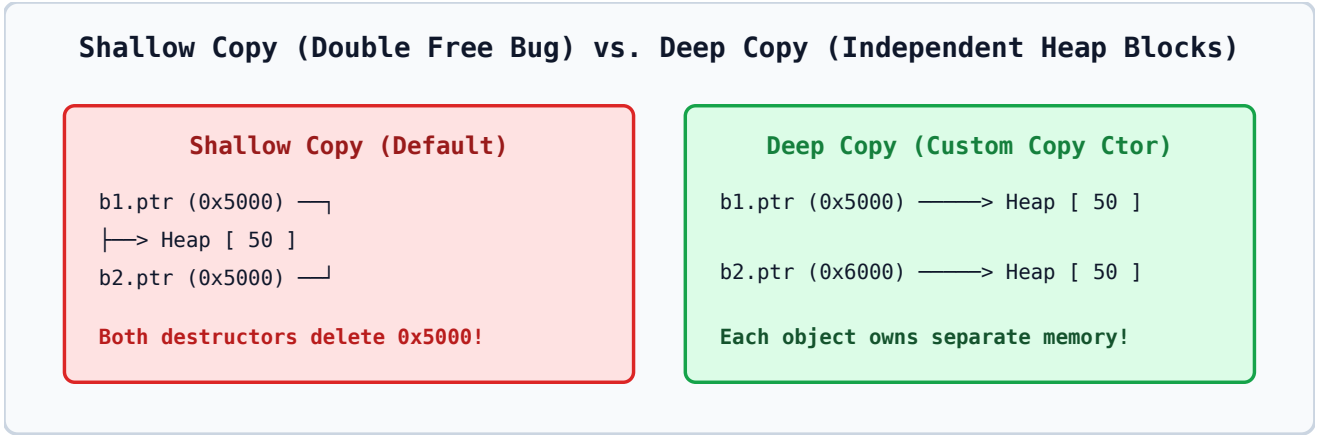
1. Properties of a Constructor

- Has the **exact same name** as the class.
- Has **no return type** (not even `void`).
- Invoked automatically when an object is created.
- If you define **any** parameterized constructor, the compiler **suppresses** the implicitly-declared default constructor (unless you restore it with `person() = default;`).

2. When Member Initializer Lists (`: member(val)`) Are Mandatory

1. Initializing `const` **data members** (`const int id;`).
2. Initializing **reference data members** (`int& ref;`).
3. Initializing **base classes** or **member objects** that lack a default constructor.
4. **Initialization Order Rule**: Data members are always initialized in the **exact order they are declared inside the class**, **not** the order written in the member initializer list!

02 — Copy Constructor (`person(const person& other)`) and Shallow vs. Deep Copy



Shallow vs Deep Copy

1. Why Must the Copy Constructor Parameter Be Passed by Reference?

If a copy constructor were declared by value as `person(person other)`, calling the copy constructor to pass `other` by value would require calling the copy constructor again to copy `other`, creating **infinite recursion**! Therefore, the C++ standard makes `person(person other)` a **compile-time error** (`[class.copy.ctor]`).

2. Shallow Copy vs. Deep Copy

- **Compiler-Generated Default Copy Constructor (Shallow Copy)**: Copies each member bit-by-bit/member-by-member. If a class owns a raw heap pointer `int* data`, both objects end up pointing to the **exact same heap block**, causing **Double Deletion UB** when both destructors run!
- **User-Defined Deep Copy Constructor**: Allocates a **separate new heap block** (`data = new int(*other.data);`) so each object owns independent memory.

03 — Destructors (`~ClassName()`) and the Rule of Three / Five / Zero

1. Properties of a Destructor

- Named `~ClassName()`, takes **no parameters**, cannot be overloaded, and has **no return type**.
- Called automatically when a stack object leaves scope or when `delete` is called on a heap object.

2. The Rule of Three, Rule of Five, and Rule of Zero

- **Rule of Three**: If a class requires a custom **Destructor**, **Copy Constructor**, or **Copy Assignment Operator (`operator=`)**, it almost certainly requires all three.
- **Rule of Five (C++11–C++23)**: Adds the **Move Constructor** (`T(T&&) noexcept`) and **Move Assignment Operator** (`T& operator=(T&&) noexcept`).
- **Rule of Zero**: Prefer using standard RAII members (`std::string`, `std::vector`, `std::unique_ptr`) so you don't need to write any custom destructor or copy/move operations at all!

Visual Concept & Memory Diagrams (13 CONSTRUCTORS AND DESTRUCTORS)

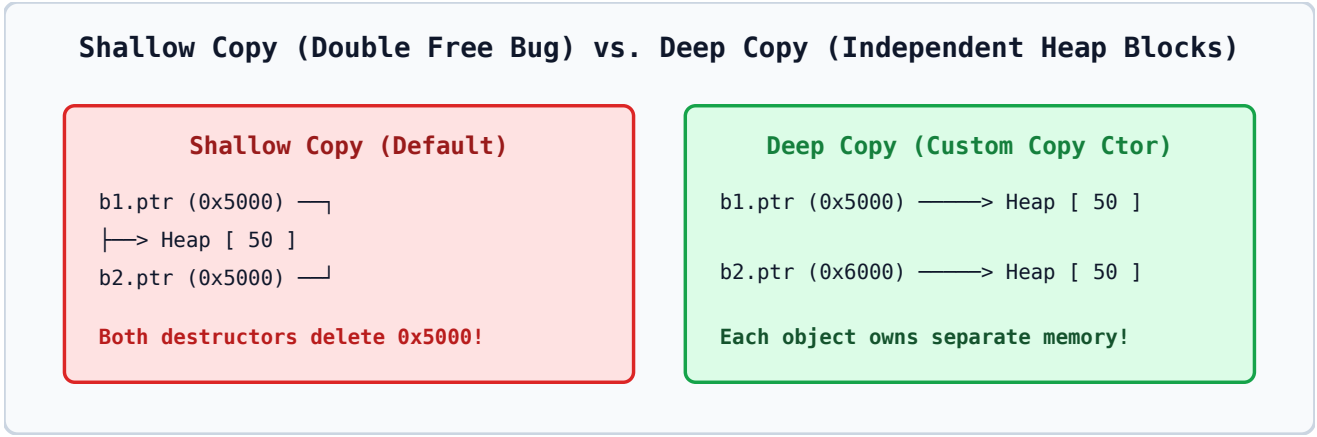


Diagram: diagram 01 shallow vs deep copy

Annotated C++ Programs (13 CONSTRUCTORS AND

DESTRUCTORS)

Program: `01_default_parameterized_and_copy_ctor.cpp`

C++ SOURCE FILE • 01_default_parameterized_and_copy_ctor.cpp

```cpp
// Program Title: Default, Parameterized, and Copy Constructors in Classroom 'person' Class
// Topic: Constructors and Destructors
// Purpose: Demonstrate all three constructor types and destructor invocation order.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_constructors_types_and_initializer_lists.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class person {
private:
    int age;

public:
    person() {
        age = 18;
        cout << "Default constructor (age=" << age << ")" << endl;
    }

    person(int a) {
        age = a;
        cout << "Parameterized constructor (age=" << age << ")" << endl;
    }

    person(const person& other) {
        age = other.age;
        cout << "Copy constructor (copied age=" << age << ")" << endl;
    }

    ~person() {
        cout << "Destructor for age=" << age << endl;
    }
};

int main() {
    person p1;
    person p2(25);
    person p3 = p2;
    return 0;
}

/*
Expected Deterministic Output:
Default constructor (age=18)
Parameterized constructor (age=25)
Copy constructor (copied age=25)
Destructor for age=25
Destructor for age=25
```

```
Destructor for age=18
*/
```

Program: `02_deep_copy_classroom_buffer.cpp`

C++ SOURCE FILE • 02_deep_copy_classroom_buffer.cpp

```cpp
// Program Title: Deep Copy Constructor Preventing Double Deletion
// Topic: Constructors and Destructors — Deep Copy vs. Shallow Copy
// Purpose: Demonstrate a classroom class managing heap memory safely with a Deep Copy constructor.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_copy_constructor_shallow_vs_deep_copy.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class number_box {
private:
    int* ptr;

public:
    number_box(int val) {
        ptr = new int(val);
    }

    // Deep Copy Constructor: allocates a separate int on the heap
    number_box(const number_box& other) {
        ptr = new int(*(other.ptr));
    }

    void set_val(int val) {
        *ptr = val;
    }

    void display() const {
        cout << "Value: " << *ptr << endl;
    }

    ~number_box() {
        delete ptr;
    }
};

int main() {
    number_box b1(50);
    number_box b2 = b1; // Invokes Deep Copy constructor

    b2.set_val(99);     // Modifies only b2's heap integer, leaving b1 untouched!

    b1.display();
    b2.display();

    return 0;
}

/*
Expected Deterministic Output:
Value: 50
```

```
Value: 99
*/
```

Program: `03_copy_ctor_by_value_compile_error.cpp`

C++ SOURCE FILE • 03_copy_ctor_by_value_compile_error.cpp

```cpp
// INTENTIONAL COMPILE-TIME ERROR DEMONSTRATION
// Program Title: Copy Constructor Declared with Pass-by-Value Parameter
// Topic: Constructors and Destructors — Copy Constructor Signature Rule
// Purpose: Show why 'person(person other)' is rejected by the C++ compiler.
// Target C++ Standard: C++23
// Related Theory File: ../01_THEORY/02_copy_constructor_shallow_vs_deep_copy.md
// Program Classification: Category B — Intentional compilation-error demonstration

#include <iostream>
using namespace std;

class person {
private:
    int age;

public:
    person(int a = 0) : age(a) {}

    // COMPILE ERROR: invalid constructor; you probably meant 'person (const person&)'
    person(person other) {
        age = other.age;
    }
};

int main() {
    person p1(20);
    return 0;
}

/*
Why Compilation Fails:
According to ISO C++ [class.copy.ctor], a constructor for class X is ill-formed if its first
parameter
is of type X (by value) and there are no other parameters without default arguments, because
passing X
by value would recursively invoke the copy constructor infinitely.
*/
```

Program: `04_member_initializer_list_const_ref.cpp`

C++ SOURCE FILE • 04_member_initializer_list_const_ref.cpp

```cpp
// Program Title: Initializing const and Reference Data Members via Member Initializer List
// Topic: Constructors and Destructors — Member Initializer Lists
// Purpose: Show how const and reference members are initialized using ': roll(r),
score_ref(s)'.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_constructors_types_and_initializer_lists.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class student {
private:
    const int roll;
    int& score_ref;

public:
    student(int r, int& s) : roll(r), score_ref(s) {
    }

    void display() const {
        cout << "Roll: " << roll << ", Score: " << score_ref << endl;
    }
};

int main() {
    int external_score = 95;
    student s(101, external_score);
    s.display();
    return 0;
}

/*
Expected Deterministic Output:
Roll: 101, Score: 95
*/
```

Conceptual Questions — Module 13: Constructors and Destructors

Q1. Why must the copy constructor parameter be passed by reference (`const T&`) rather than by value (`T`)? **Q2.** List three situations where a Member Initializer List is mandatory rather than assigning inside the constructor body.

Debugging Questions — Module 13

Q4. Why does `person p;` fail to compile if `class person` defines `person(int a) { age = a; }` and no other constructors?

Predict the Output Questions — Module 13

Q3. Predict the output of:

```cpp
#include <iostream>
using namespace std;
class test {
    int a;
    int b;
public:
    test(int x) : b(x), a(b + 10) {
        cout << a << " " << b << endl;
    }
};
```

(Hint: In what order are `a` and `b` initialized?)

Programming Questions — Module 13

Q5. Write a C++23 classroom program implementing the Rule of Three (`Destructor` , `Copy Constructor` , and `Copy Assignment Operator`) for a `dynamic_array` class.

Detailed Solutions — Conceptual Questions (Module 13)

- **Q1 Solution**: Passing the parameter by value would require copying the argument, which calls the copy constructor itself, leading to infinite recursion.
- **Q2 Solution**: (1) `const` non-static data members, (2) reference data members (`T&`), and (3) base classes or member subobjects that lack a default constructor.

Detailed Solutions — Predict the Output & Debugging (Module 13)

- **Q3 Solution**: **Undefined Behavior!** Because `int a;` is declared **before** `int b;` inside `class test`, `a(b + 10)` is evaluated **first** while `b` is still uninitialized! Always write member initializer lists in the exact declaration order of the data members.
- **Q4 Solution**: Defining any user-declared constructor suppresses the compiler-generated default constructor. Fix: add `person() = default;` or provide a default argument `person(int a = 0)`.

Detailed Solutions — Programming Questions (Module 13)

CPP

```cpp
#include <iostream>
using namespace std;

class dynamic_array {
private:
    int size;
    int* arr;
public:
    dynamic_array(int s) : size(s), arr(new int[s]{}) {}

    dynamic_array(const dynamic_array& other) : size(other.size), arr(new int[other.size]) {
        for (int i = 0; i < size; i++) arr[i] = other.arr[i];
    }

    dynamic_array& operator=(const dynamic_array& other) {
        if (this != &other) {
            delete[] arr;
            size = other.size;
            arr = new int[size];
            for (int i = 0; i < size; i++) arr[i] = other.arr[i];
        }
        return *this;
    }

    ~dynamic_array() { delete[] arr; }
};

int main() {
    dynamic_array a(3);
    dynamic_array b = a;
    return 0;
}
```